



CS-351: Compiler Construction

(Spring 2026)



Lecture 37

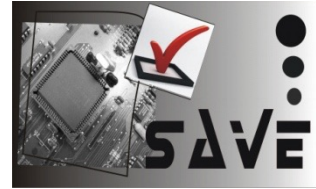
Adnan Rashid

Email: adnan.rashid@seecs.edu.pk

Web: <https://adrashid.github.io/>



Recap



- Optimization in Loops



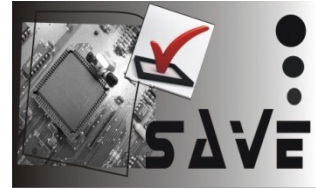
Outline



- Intermediate Code Generation
 - AST, DAG and TAC



Lecture's Content



Compiler: Principles, Techniques and Tools (2nd Edition)

by

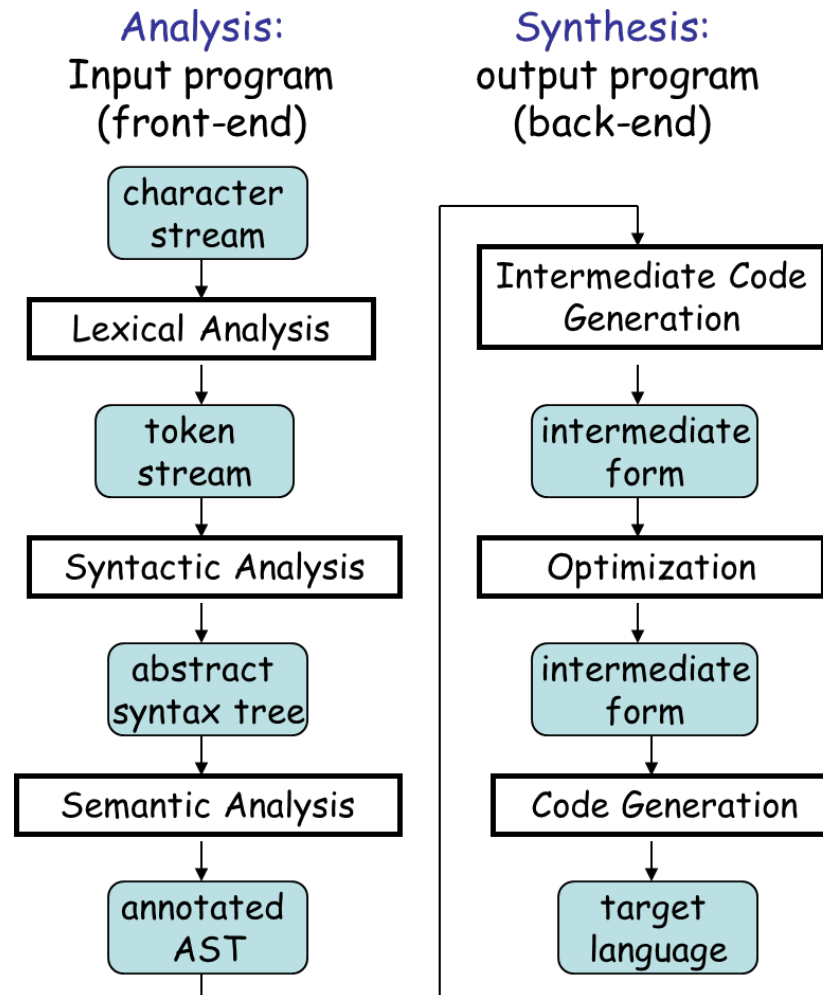
Alfred V. Aho et al.

Sections 6.1, 6.2 and 8.5

(Pages 358, 363 and 533 onwards)



Compiler Design





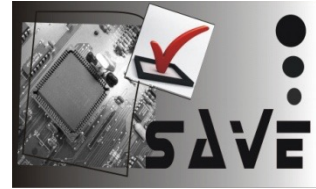
Why do IR Generation?



- Simplify certain optimizations
 - Machine code has many constraints that inhibit optimization
 - Working with an intermediate language makes optimizations easier and clearer



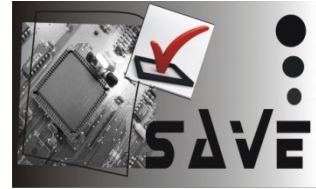
Why do IR Generation?



- Have many front-ends into a single back-end
 - GCC can handle C, C++, Objective-C, Fortran, Ada, and many other languages
 - Each front-end translates source to the GENERIC language
- Have many back-ends from a single front-end
 - Do most optimization on intermediate representation before emitting code targeted at a single machine



Designing a Good IR



- Need to balance needs of high-level source language and low-level target language
- Too high level: can't optimize certain implementation details
- Too low level: can't use high-level knowledge to perform aggressive optimizations
- Often have multiple IRs in a single compiler
- Example: Architecture of GCC



Thank you!